DOCUMENTAZIONE SOFTWARE / EDIZIONE ITALIANA

SPEECHSHARE PHON MODULAR 4

Manuale tecnico e d’uso

Architettura DSP · PATCH ASIO · gain sharing · collaudo

VERSIONE DI RIFERIMENTO: 0.3.1 - AUTO SESSION SLEEP

BACKSTAGE / DOCUMENTAZIONE DI PROGETTO

23 OTTOBRE 2026

Informazioni sul documento

Oggetto. Manuale della variante standalone Windows **SpeechShare PHON Modular 4 Sleep v0.3.1**. Il testo descrive le funzioni presenti nei sorgenti disponibili in data 23 ottobre 2026, i relativi controlli e le procedure di installazione e prova.

Base documentale. Archivio `SpeechShare_Standalone_PHON_Modular_4Session_v0.3.1_SLEEP_SOURCE.zip`: file C++, CMake e README inclusi. Le spiegazioni matematiche sono derivate in prevalenza da `Insert/InsertDSP.cpp` e `Insert/SharedPool.cpp`.

> **Stato del software:** prototipo tecnico per prove offline. Il routing a due sessioni e i percorsi mono/stereo sono stati riferiti come funzionanti nei test dell'utente sulla variante precedente. In questa sessione sono stati compilati ed eseguiti **7 test C++ locali su 7** della v0.3.1; NON sono stati eseguiti qui compilazione MSVC/JUCE, collaudo ASIO, stress test prolungati o misura della latenza round-trip.

Convenzioni. "Partecipante" = sorgente logica mono oppure coppia stereo sottoposta a un singolo AutoGain. "Sessione" = banco di 8 partecipanti con gruppi A/B/C. "Tap" = segnale interno post-elaborazione usato da un'altra sessione. "Porta" = canale I/O presentato al callback del driver.

Destinatari. Fonici broadcast, operatori di regia, tecnici audio, sviluppatori C++/DSP e personale di collaudo. Le indicazioni sono intenzionalmente conservative rispetto alla sicurezza dei segnali e all'impiego live.

Indice dei contenuti

Informazioni sul documento 2

1. Panoramica: che cos’è SpeechShare 4

2. Requisiti e installazione 4

3. Prima accensione e procedura sicura 5

4. Interfaccia principale 6

5. Configurazione driver e buffer ASIO 7

6. PATCH MATRIX: ingressi 7

7. PATCH MATRIX: uscite e mix di gruppo 8

8. Routing a quattro sessioni: esempi 9

9. Gestione mono/stereo 10

10. Comandi per partecipante e per sessione 10

11. PHON: macro di risposta 11

12. Virtual KEY: riferimento che non va in uscita 12

13. DSP: catena di elaborazione 12

14. DSP: gain sharing, tempi e normalizzazione 13

15. Architettura tecnica e callback audio 14

16. Auto Session Sleep e Callback Load 15

17. Salvataggio, preset e ripristino 16

18. Collaudi funzionali consigliati 16

19. Prestazioni, latenza e capacità 18

20. Diagnostica dei problemi comuni 19

21. Limiti noti e scelte progettuali 19

22. Glossario operativo 20

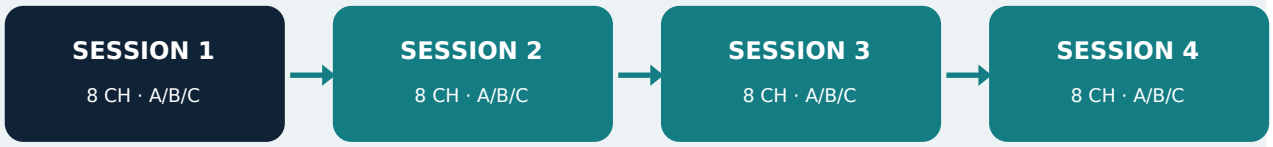
23. Riferimenti ai file sorgente e verifiche 20

24. Scheda operativa rapida 21

1. Panoramica: che cos’è SpeechShare

SpeechShare PHON Modular 4 è un **automixer a gain sharing** eseguibile come applicazione standalone Windows, con ingresso e uscita attraverso un dispositivo gestito da JUCE (inclusi i driver ASIO disponibili). Non richiede una DAW durante l’esecuzione standalone.

- Quattro sessioni indipendenti, numerate 1-4.
- Otto partecipanti per sessione: **32 partecipanti totali** (una coppia stereo occupa un partecipante).
- Tre gruppi di automix per sessione: A, B e C.
- Per ciascun partecipante: Mono o Stereo, Group, Mode, Mute, Solo, Weight e meter.
- Per sessione: Classic o Smart, controlli master, Virtual KEY e PHON.
- Uscite ASIO assegnabili a singoli partecipanti o a mix di gruppo.
- Riuso interno di singoli canali o gruppi di una sessione precedente in una successiva.
- Auto Session Sleep e indicatore Callback Load introdotti nella v0.3.1.



Routing consentito solo verso sessioni successive; link diretti e salti ammessi

Regola fondamentale: le sessioni non sono quattro unità fisiche in cascata obbligatoria. Possono operare indipendentemente o alimentarsi fra loro, ma soltanto in direzione crescente (per esempio 1→4 o 2→3→4). Le frecce di Overview mostrano la direzione consentita, non un PATCH realmente attivo.

Riferimento sorgente: App/ModularPatch.h; App/StandaloneAudio.cpp; App/Main.cpp

2. Requisiti e installazione

2.1 Requisiti di compilazione

Componente	Funzione / requisito
Windows x64	Sistema di destinazione dell'app standalone.
JUCE 8	Framework grafico e audio; sorgente/configurazione JUCE accessibile dal progetto.
CMake ≥ 3.22	Generazione del progetto e orchestrazione della build.
Visual Studio 2026 + MSVC C++	Compilatore target dello script distribuito; generatore CMake "Visual Studio 18 2026".
Driver ASIO	Opzionale come categoria di driver, necessario per prove a latenza ridotta con la specifica interfaccia.
Hardware audio / rete	Per Dante Virtual Soundcard è necessaria una rete Dante funzionante e sincronizzata.

Lo script di build usa per impostazione predefinita `C:\JUCE`, configurabile tramite il parametro `-JuceRoot`.

2.2 Compilazione da sorgenti

Estrarre lo ZIP in una nuova directory. Aprire PowerShell nella cartella principale, quella contenente `CMakeLists.txt` e `build_live_windows_2026.ps1`. Eseguire:

```
powershell -ExecutionPolicy Bypass -File ".\build_live_windows_2026.ps1"
```

Lo script configura la build Release in `build-modular-vs2026`, compila l'applicazione e tutti i test, poi esegue CTest. L'eseguibile si trova normalmente in:

```
build-modular-vs2026\SpeechSharePHONModular_artefacts\Release
```

Controllo di compilazione: il messaggio "100% tests passed" conferma soltanto che i test automatizzati C++ hanno avuto esito positivo. Non certifica routing ASIO reale, interoperabilità con Dante o assenza di dropout.

Riferimento sorgente: CMakeLists.txt; build_live_windows_2026.ps1; README_SESSION_SLEEP_v0.3.1.md

3. Prima accensione e procedura sicura

Alla partenza il motore audio rimane **disarmato**. Anche con dispositivo audio disponibile, le uscite vengono pulite finché non si preme RUN AUDIO.

Sequenza consigliata

- Avviare l'app senza collegare le uscite a un impianto di diffusione o a un percorso broadcast critico.
- Aprire **PATCH / ASIO**: l'apertura disarma automaticamente l'audio.
- Nel tab **AUDIO DRIVER / BUFFER** scegliere driver, dispositivo, sample rate e buffer; abilitare i canali I/O necessari.
- Nel tab **PATCH MATRIX** assegnare gli ingressi della Session 1 e disattivare le destinazioni hardware non necessarie.

- Premere **SAVE PATCH**. Controllare che le sessioni e i gruppi desiderati siano configurati.
- Tenere disattivata KEY MASTER durante il primo test di trasparenza del routing. Usare Classic e PHON 50.
- Abbassare i livelli di ascolto, poi premere **RUN AUDIO**. Testare una sorgente alla volta.
- Premere **STOP AUDIO** prima di una nuova modifica al PATCH o al dispositivo.

Attenzione: il salvataggio del PATCH non riavvia automaticamente l'elaborazione. Dopo ogni accesso al pannello PATCH / ASIO è necessario riarmare con RUN AUDIO.

4. Interfaccia principale

4.1 Barra superiore

Elemento	Funzione
PATCH / ASIO	Apre impostazioni device e matrici input/output; disarma l'audio.
RUN AUDIO / STOP AUDIO	Arma o disarma l'elaborazione e il passaggio audio.
SESSION 1…4	Mostra l'editor dell'automixer selezionato.
OVERVIEW	Mostra i quattro moduli e lo stato ACTIVE/SLEEP.
Device label	Mostra driver, nome dispositivo, sample rate, buffer, porte I/O e durata di callback nominale.
CALLBACK LOAD / PEAK	Tempo di elaborazione audio rapportato alla durata disponibile del buffer.
SESSIONS x/4 AWAKE	Numero di sessioni con sorgenti utilizzabili al callback.

4.2 Session editor

Ogni sessione mostra gli otto partecipanti della propria banca. L'editor eredita la grafica del processore Insert: meter INPUT/AUTO GAIN, Weight, MUTE/SOLO, gruppi A/B/C e selezione della modalità del canale.

La manopola PHON e le impostazioni Virtual KEY sono a destra. Il selettore interno delle sessioni nel processore è disabilitato nella variante modulare: per navigare si utilizzano i pulsanti esterni SESSION 1-4.

Sono disponibili i comandi di sessione **MASTER MUTE**, **MASTER BYPASS**, **MASTER dB**, le scelte **CLASSIC/SMART** e i controlli di copia/incolla preset tramite appunti, presenti nel codice della GUI.

4.3 Overview

Overview è una rappresentazione delle quattro sessioni. I riquadri sono cliccabili e mostrano ACTIVE o SLEEP quando l'audio gira. Le frecce non rappresentano gli effettivi collegamenti: la lista delle connessioni va controllata nel PATCH MATRIX.

Riferimento sorgente: App/Main.cpp; Insert/InsertPluginEditor.cpp

5. Configurazione driver e buffer ASIO

Il pannello **AUDIO DRIVER / BUFFER** utilizza il selettore dispositivi JUCE. Le operazioni disponibili dipendono dal driver scelto: alcune interfacce espongono pannelli dedicati per frequenza, buffer e sincronizzazione.

5.1 Sample rate

In una rete broadcast operante a 48 kHz, impostare 48.000 Hz nel dispositivo/driver, non soltanto in SpeechShare. In presenza di Dante Virtual Soundcard è necessario che la rete Dante sia disponibile e sincronizzata.

5.2 Dimensione buffer

Buffer	Durata a 48 kHz	Impiego suggerito
64 campioni	1,33 ms	Stress test dopo aver stabilizzato routing e driver.
128 campioni	2,67 ms	Test a bassa latenza su macchina adeguata.
256 campioni	5,33 ms	Punto di partenza prudente per il collaudo.
512 campioni	10,67 ms	Diagnosi di problemi di stabilità/dropout.
1024 campioni	21,33 ms	Debug; in genere troppo elevato per molti impieghi live.

La durata di un blocco è **B / f_s**: non coincide con la latenza reale ingresso→uscita, che comprende driver, convertitori, rete e ulteriori buffering.

5.3 Abilitazione delle porte

Se il dispositivo rileva numerosi canali ma i menu PATCH non producono audio, verificare che le rispettive porte siano effettivamente abilitate. Il tasto **ENABLE ALL ASIO CHANNELS** richiede l’attivazione delle porte enumerate dal device (fino a 128 ingressi e 128 uscite).

> **Nota implementativa:** nel callback i canali sono indicizzati in base agli array di porte presentati da JUCE. Quando si usano maschere di canali abilitate in modo non contiguo, la corrispondenza tra numeri in PATCH e numerazione fisica deve essere verificata con segnali di prova.

Riferimento sorgente: App/Main.cpp; App/StandaloneAudio.cpp; App/ModularPatch.h

6. PATCH MATRIX: ingressi

La colonna sinistra di PATCH MATRIX assegna una sorgente al partecipante di destinazione. La Session 1 può ricevere solo ingressi fisici ASIO. Le Sessioni 2-4 possono ricevere anche singoli canali o gruppi di **sessioni precedenti**.

Tipologia input	Descrizione
OFF	Partecipante escluso dal flusso e, quando non elaborato, dal gain sharing.

Tipologia input	Descrizione
ASIO IN N (MONO)	Ingresso singolo del driver.
ASIO IN N+N+1 (STEREO)	Coppia di porte consecutive, con un solo controllo automatico linked.
Sx CH y MONO	Tap post-elaborazione di un partecipante in una sessione precedente, letto come mono.
Sx CH y STEREO	Come sopra, utilizzato come coppia stereo.
Sx GROUP A/B/C MONO	Mix post-automixer del gruppo, convertito a mono.
Sx GROUP A/B/C STEREO	Mix post-automixer del gruppo in formato stereo.

6.1 Regole di validità

- Session 1 non accetta sorgenti interne.
- Session 2 può ricevere soltanto tap di Session 1.
- Session 3 può ricevere tap di Session 1 o Session 2.
- Session 4 può ricevere tap di Session 1, 2 o 3.
- Non sono consentiti collegamenti verso la stessa sessione, connessioni a ritroso o anelli di feedback.
- Un ingresso ASIO mono può alimentare più partecipanti; l’esclusività riguarda le uscite hardware, non gli ingressi.

6.2 RESET SAFE 1:1

Ripristina **Session 1 CH 1-8** rispettivamente su **ASIO IN 1-8** e **ASIO OUT 1-8** in mono; Sessioni 2, 3 e 4 restano OFF, come tutte le uscite di gruppo. Se il driver ha soltanto due uscite, occorre liberare le assegnazioni 3-8 o comunque le destinazioni necessarie al nuovo test.

Riferimento sorgente: App/ModularPatch.h; App/Main.cpp

7. PATCH MATRIX: uscite e mix di gruppo

7.1 Tipologie di destinazione

Per ogni singolo partecipante e per ciascun mix A/B/C di ogni sessione, l’output può essere OFF, **ASIO OUT N (MONO)** oppure **STEREO OUT N+N+1**.

Il mix di gruppo è la **somma audio post-automixer e post-gain** dei partecipanti assegnati al gruppo. Non è unicamente un valore di detector. Sono disponibili 4 × 3 = **12 bus di gruppo** e 32 tap individuali.

7.2 Esclusività delle porte fisiche

Una porta ASIO di uscita non può essere assegnata contemporaneamente a due destinazioni SpeechShare. Se una coppia stereo occupa OUT 1+2, non è possibile assegnare OUT 2 a un altro partecipante o bus. Il PATCH rifiuta l’operazione e conserva l’assegnazione precedente.

Consiglio operativo: per ascoltare solo l'uscita finale di una catena S1→S4, assegnare una porta hardware soltanto alla Session 4 e lasciare OFF gli output fisici delle sessioni precedenti. I tap interni rimangono disponibili anche se l'output hardware è OFF.

7.3 Livelli e clipping

Non è presente un limiter sui mix di gruppo. Se più segnali correlati sono sommati, la loro ampiezza può superare 0 dBFS. Usare gain staging conservativo e verificare i livelli degli output con strumenti esterni. I controlli MASTER dB e CHANNEL LEVEL sono post-automix e possono aumentare il rischio di saturazione.

Riferimento sorgente: App/StandaloneAudio.cpp; App/ModularPatch.h; Insert/InsertPluginProcessor.cpp

8. Routing a quattro sessioni: esempi

8.1 Catena mono completa

```
S1 CH 1  <- ASIO IN 1 (MONO)
S2 CH 1  <- S1 GROUP A (MONO)
S3 CH 1  <- S2 GROUP A (MONO)
S4 CH 1  <- S3 GROUP A (MONO)
S4 CH 1  -> ASIO OUT 1 (MONO)
```

Assegnare CH 1 di ogni sessione al gruppo A e impostare tutti i canali in AUTO. Lasciare OFF le altre uscite hardware. Per la prova iniziale disattivare KEY MASTER, selezionare CLASSIC e impostare PHON a 50.

8.2 Catena stereo

```
S1 CH 1  <- ASIO IN 1+2 (STEREO)
S2 CH 1  <- S1 GROUP A (STEREO)
S3 CH 1  <- S2 GROUP A (STEREO)
S4 CH 1  <- S3 GROUP A (STEREO)
S4 CH 1  -> STEREO OUT 1+2
```

La stessa decisione di AutoGain viene applicata ai due lati della sorgente stereo; i bus e i tap conservano un formato a due canali. La coppia fisica 1+2 resta interamente occupata dalla destinazione della Session 4.

8.3 Mix paralleli e sottogruppi

Un esempio broadcast è: Session 1 per sei lavalier, Session 2 per due microfoni di servizio, Session 3 per una scena indipendente e Session 4 come stage di automix finale che riceve il **GROUP A di Session 1**, il **GROUP B di Session 2** e un tap di Session 3. Ogni ingresso di Session 4 è un partecipante a sua volta. Valutare l'opportunità di sottoporre di nuovo i segnali già automixati a gain sharing.

Nota sul gain staging a cascata: i passaggi mono→bus stereo→mono applicano le conversioni previste nel codice e possono introdurre attenuazioni aggiuntive. Per una catena mono che transita attraverso mix di gruppo, misurare il livello a ogni salto; non assumere automaticamente unità di guadagno.

Riferimento sorgente: App/StandaloneAudio.cpp; App/ChannelFormat.h

9. Gestione mono/stereo

Il formato mono/stereo appartiene a ciascuna **sorgente PATCH**, non all'intera sessione. Una coppia stereo conta come un partecipante: non consuma due slot del pool.

Percorso	Trattamento implementato
Stereo nativo	L e R restano separati; l'RMS del detector usa la media della potenza di entrambi. Stesso guadagno applicato a L/R.
Stereo → mono	M = (L + R) / 2. Verificare compatibilità di fase del materiale L/R.
Mono → stereo	L = R = M / √2 (≈ -3,01 dB per lato).
Mono → bus di gruppo	Il singolo mono è distribuito a L e R del bus con fattore 1/√2.
Stereo → bus di gruppo	Somma separata dei campioni L/R.
Stereo OUT → mono fisico	Il dispositivo riceve la media delle due componenti L/R.

I menu consentono coppie ASIO **consecutive** (1+2, 2+3, ecc. entro le porte disponibili). Non è prevista la selezione indipendente dei numeri L e R della stessa sorgente stereo.

> **Misura consigliata:** testare anche segnali stereo anti-correlati: la conversione stereo→mono tramite media può annullare in parte o completamente componenti fuori fase. Questo è un comportamento matematico previsto e non un guasto dell'automixer.

Riferimento sorgente: App/ChannelFormat.h; App/StandaloneAudio.cpp; Insert/InsertDSP.cpp

10. Comandi per partecipante e per sessione

10.1 MODE per partecipante

Modalità	Comportamento
AUTO	Partecipa allo score del proprio gruppo e riceve il gain sharing.
MAN	Non partecipa alla competizione AUTO; il guadagno del DSP tende a unità, con gli smoothing previsti.
BYP	Esclude la decisione di automix e forza il guadagno base a unità; restano validi MUTE e i gain post-DSP.
MUTE / LEGACY	Imposta il guadagno target a zero; il canale è escluso dal set AUTO.

10.2 MUTE, SOLO e Weight

MUTE è distinto da AUTO e BYP: esclude il contributo al gain sharing e introduce una breve rampa locale di chiusura (circa 3 ms). **SOLO** agisce entro la sola sessione, non sul PATCH delle altre sessioni. **Weight** modifica la sensibilità del canale nell'assegnazione del guadagno, senza coincidere con il livello di uscita.

Intervalli del codice: Weight da -24 a +24 dB, reset a 0 dB. Il livello di output per canale gestito dal processore è configurabile da -90 a +12 dB; la GUI principale mostra soprattutto il Weight, mentre altre funzioni possono essere accessibili dalle impostazioni di sessione/preset.

10.3 Comandi globali

MASTER MUTE silenzia i partecipanti della sessione. MASTER BYPASS esclude l'automix per la sessione. MASTER dB regola il livello post-DSP di tutti i partecipanti. CLASSIC/SMART modifica la modalità di detector dei partecipanti associati alla sessione.

Riferimento sorgente: Insert/InsertPluginProcessor.cpp; Insert/InsertPluginEditor.cpp; Insert/SharedPool.cpp

11. PHON: macro di risposta

PHON è una macro **0–100** che aggiorna Strength, Attack e Release per la sessione attiva. Non aggiunge un blocco DSP e non introduce intenzionalmente lookahead o memoria di ritardo audio.

PHON	Strength	Attack	Release	Profilo
0	0,60	28 ms	350 ms	Morbido
50	1,00	12 ms	160 ms	Neutro
100	2,20	4 ms	70 ms	Aggressivo

Tra 0 e 50 e tra 50 e 100 i parametri sono calcolati per **interpolazione lineare a tratti**. Il doppio clic sulla manopola riporta PHON a 50. RMS e HPF del detector **non vengono riscritti dalla rotazione**: normalmente restano a 100 ms e 100 Hz, salvo configurazioni già differenti.

$$v(p) = v0 + (v1 - v0) \cdot t$$

Interpolazione del parametro; t = p/50 oppure (p - 50)/50

Aumentando PHON cresce la selettività matematica dello score e diventano più rapide le variazioni del guadagno. Non è però una soglia gate e non corrisponde a una percentuale lineare di attenuazione.

Uso consigliato: PHON 50 come riferimento; spostarsi verso 0 quando si desiderano transizioni più morbide e verso 100 quando occorre una separazione più rapida fra sorgenti. Validare sempre all'ascolto con parlato sovrapposto e livelli realistici.

Riferimento sorgente: Insert/PhonMacro.h; Insert/InsertPluginEditor.cpp

12. Virtual KEY: riferimento che non va in uscita

Ogni sessione dispone di tre KEY virtuali, una per gruppo A/B/C. KEY MASTER abilita globalmente il sistema della sessione; il singolo KEY può poi essere abilitato, impostato come Sine o Pink, con frequenza (per Sine) e livello in dB.

Parametro	Intervallo / default
KEY MASTER	Disabilitato per impostazione iniziale.
KEY A/B/C ON	Abilitato nella configurazione del gruppo, ma inerte se KEY MASTER è OFF.
Generator	Sine o Pink; default Sine.
Frequency	10–20.000 Hz; default 40 Hz (solo Sine).
Level	-96…0 dB; default -56 dB.

La KEY viene generata e misurata **solo nel detector**. Non occupa un canale audio, non appare sulle uscite e non forma un partecipante del pool. Aggiunge però un contributo virtuale al denominatore dello score, riducendo la tendenza di un microfono isolato a recuperare guadagno unitario nei silenzi.

$$G_i = \sqrt{\frac{S_i}{(\Sigma S_j + K)}}$$

K = potenza detector KEY elevata a Strength, se abilitata

La KEY non è un gate a soglia e non elimina necessariamente il rumore di fondo. In particolare, una KEY con livello eccessivo può produrre attenuazioni indesiderate anche mentre si parla.

Riferimento sorgente: Insert/InsertDSP.cpp; Insert/SharedPool.h; Insert/InsertPluginEditor.cpp

13. DSP: catena di elaborazione

Per ogni partecipante attivo il percorso effettivo può essere letto così: ingresso ASIO/tap → buffer mono/stereo → detector HPF → misura RMS e Smart opzionale → score → condivisione score del gruppo → guadagno target → Attack/Release → eventuale normalizzazione Classic → MUTE → post-gain → tap individuale e mix di gruppo.

13.1 Detector RMS

$$RMS = \sqrt{\frac{1}{N} \cdot \Sigma x[n]^2}$$

Misura su una finestra temporale; 100 ms a 48 kHz = circa 4.800 campioni

Il detector precede la decisione dell’automixer; il segnale audible non viene filtrato dal detector HPF. L’HPF ha un intervallo di regolazione di 0–350 Hz, predefinito 100 Hz. L’RMS ha un intervallo di 10–500 ms.

13.2 Score Classic

$$S_i = (RMS_i \cdot 10^{(Weight_i/20)})^{(2 \cdot Strength)}$$

Il Weight modifica la potenza attribuita a una sorgente: +6 dB moltiplica circa per due il suo RMS ponderato. Strength enfatizza matematicamente le differenze fra partecipanti.

13.3 Score Smart (sperimentale)

$$P = \max(0, 0.65 \cdot Pslow + 0.35 \cdot Pfast - 1.6 \cdot Pnoise) + 0.001 \cdot Pslow$$

Lo score Smart è calcolato come **(sqrt(P) × fattore Weight)^(2×Strength)**. Le componenti principali sono la potenza RMS lenta, la potenza veloce e una stima adattiva del rumore. Smart è un'euristica di detector, non un algoritmo certificato di separazione della voce.

Nel codice i tempi caratteristici del fast detector sono circa 9 ms in salita e 60 ms in discesa; la stima del rumore segue più rapidamente le diminuzioni (circa 350 ms) e lentamente gli aumenti (circa 8 s).

Riferimento sorgente: Insert/InsertDSP.cpp; Insert/InsertDSP.h

14. DSP: gain sharing, tempi e normalizzazione

14.1 Formula di condivisione

$$Gtarget_i = \sqrt{ S_i / (\Sigma S_j + K) }$$

La somma al denominatore considera gli score dei microfoni AUTO nello stesso gruppo della stessa sessione, più la KEY virtuale se abilitata. Il canale MAN/BYP/MUTE non partecipa a questa competizione.

Senza KEY e in condizioni matematiche ideali, la somma dei quadrati dei gain target dei partecipanti AUTO è pari a 1. Non ne consegue una somma acustica costante in presenza di segnali correlati o di smoothing diversi.

14.2 Esempio numerico

Canale	RMS	Score (Strength 1)	Guadagno target
LAV 1	0,10	0,0100	0,894 = -0,97 dB
LAV 2	0,05	0,0025	0,447 = -6,99 dB

I numeri rappresentano i target teorici prima dei filtri temporali e della normalizzazione Classic.

14.3 Smoothing Attack / Release

$$G[n] = Gtarget + a \cdot (G[n-1] - Gtarget)$$

$$a = \exp(-1 / (f_s \cdot T))$$

Attack se il guadagno sale; Release se il guadagno scende

T è una costante di tempo, non un ritardo d’attesa. Dopo una costante di tempo un filtro esponenziale percorre circa il 63% dello spostamento verso il nuovo target.

14.4 Normalizzazione Classic

$$Ntarget = \sqrt{ Ptarget / \Sigma Graw_j^2 }$$

In Classic un coefficiente di normalizzazione cerca di stabilizzare la potenza complessiva dei guadagni; il coefficiente viene smussato con circa 45 ms. In Smart non viene applicata la stessa normalizzazione Classic.

Limite strutturale: il SharedPool scambia score e rawGain mediante snapshot atomici fra istanze processate in successione. La normalizzazione e la competizione non sono quindi sample-accurate come in un motore centralizzato che calcoli tutti i detector prima di tutti i gain. Questa è una caratteristica nota del prototipo.

Riferimento sorgente: Insert/InsertDSP.cpp; Insert/SharedPool.cpp

15. Architettura tecnica e callback audio

15.1 Moduli principali

File / componente	Responsabilità
App/Main.cpp	Finestra standalone, navigazione sessioni, PATCH ASIO, UI di stato.
App/StandaloneAudio.cpp	Callback dispositivo, elaborazione sequenziale delle sessioni, buffer e uscite.
App/ModularPatch.h	Grafo di connessione e convalida degli input/output mono/stereo.
App/SessionActivity.h	Valutazione delle sorgenti disponibili e maschera ACTIVE/SLEEP.
App/ChannelFormat.h	Conversioni mono↔stereo.
Insert/InsertDSP.cpp	Detector, Virtual KEY, score e AutoGain.
Insert/SharedPool.cpp	Condivisione sessione/gruppo e comandi per le istanze.
Insert/PhonMacro.h	Mappatura PHON.
Insert/InsertPluginProcessor.cpp	Integrazione DSP, stato parametri e gain post-automix.

15.2 Ordine del callback

Il callback JUCE pulisce le uscite fisiche, verifica stato ARM/READY, calcola la disponibilità delle sorgenti, gestisce i passaggi sleep/wake, poi elabora **Session 1 → Session 2 → Session 3 → Session 4**.

I blocchi grandi vengono suddivisi in segmenti di massimo **1024 campioni**. Per ogni sessione attiva vengono azzerati i bus A/B/C, processati i partecipanti disponibili e sommati i loro segnali elaborati. Le uscite fisiche dei canali e dei gruppi vengono inviate immediatamente dove assegnate.

15.3 Latenza del grafo

Poiché le sessioni precedenti vengono elaborate prima di quelle successive nello **stesso callback**, non esiste un buffer audio di ritardo intenzionale per trasportare il tap S1→S2. La latenza dell'intero sistema dipende comunque dal dispositivo e dal driver, oltre a eventuali componenti esterni.

15.4 Thread e persistenza

Il percorso audio usa principalmente buffer preallocati, snapshot atomici e controlli senza blocchi espliciti per il routing. Alcuni comandi GUI sono applicati sul message thread tramite timer di istanza a 60 Hz. Il codice di trattamento del buffer mono/stereo va ancora sottoposto ad audit e profiling delle allocazioni in callback prima di qualificare il sistema come hard real-time.

Riferimento sorgente: App/StandaloneAudio.cpp; App/StandaloneAudio.h; Insert/InsertPluginProcessor.cpp; Insert/SharedPool.h

16. Auto Session Sleep e Callback Load

16.1 Quando una sessione dorme

Una sessione è SLEEP se **nessuno dei suoi otto partecipanti ha una sorgente utilizzabile**. Sono valutati sia gli ingressi fisici ASIO presenti sia i tap individuali o di gruppo di sessioni precedenti. Il test riguarda la connessione, **non il livello audio**: una sorgente ASIO valida resta ACTIVE anche in silenzio.

Il motore salta l'elaborazione DSP della sessione inattiva e non ripulisce né invia i suoi mix inutilizzati. Rimangono però allocati i 32 processori e attivi i timer che curano i comandi UI.

16.2 Risveglio / anti-click

Alla transizione OFF→sorgente valida il detector viene azzerato e l'uscita del partecipante passa attraverso una breve rampa di circa **3 ms**. Questo non inserisce un buffer addizionale nel grafo, ma sfuma temporaneamente l'attacco del segnale.

16.3 Indicatore di carico

$$\text{Callback Load \%} = 100 \cdot T_{process} / (B / f_s)$$

Tprocess è il tempo trascorso nel callback; B è il numero di campioni del callback e f_s il sample rate. Il valore PEAK conserva un massimo a decrescita lenta (`oldPeak×0.995` per callback). I numeri rappresentano il rispetto della deadline, **non** la CPU complessiva, né la latenza round-trip.

PATCH attivo	Stato atteso
Solo S1 con ingressi ASIO validi	1/4 AWAKE
S1 con tap valido verso S2	2/4 AWAKE
Catena S1 → S2 → S3 → S4	4/4 AWAKE
Tap verso S2 OFF, senza ulteriori ingressi validi	S2 e sessioni che dipendono solo da esso tornano SLEEP

Riferimento sorgente: App/SessionActivity.h; App/StandaloneAudio.cpp; App/Main.cpp

17. Salvataggio, preset e ripristino

La v0.3.1 usa file separati dalle precedenti varianti del progetto per limitare le interferenze. Il codice salva i dati nella cartella applicativa utente SpeechShare; in Windows la posizione effettiva dipende dal percorso restituito da JUCE per `userApplicationDataDirectory`.

File XML	Contenuto
StandalonePHONModular4Sleep.xml	Stato del dispositivo audio e dei canali configurati.
StandaloneModular4SleepPatch.xml	Assegnazioni ingressi, uscite individuali e bus di gruppo.
StandaloneModular4SleepChannels.xml	Stato dei 32 processori e dei parametri salvati.

Il PATCH è salvato con nodi SOURCE e GROUP; in apertura, il codice ne verifica nuovamente le rotte e rifiuta conflitti di output. I preset dell’editor di sessione sono copiabili/incollabili tramite clipboard; i preset non sostituiscono il salvataggio del PATCH fisico.

Buona pratica di backup

- Prima di modifiche importanti premere SAVE PATCH e chiudere l’app senza audio attivo.
- Effettuare copia dei tre XML della variante 4 Sleep.
- Annotare driver, frequenza, buffer e numero di canali realmente abilitati.
- Dopo il ripristino verificare il PATCH in modo visivo prima di premere RUN AUDIO.

Riferimento sorgente: App/StandaloneAudio.cpp; Insert/InsertPluginProcessor.cpp; Insert/InsertPluginEditor.cpp

18. Collaudi funzionali consigliati

18.1 Test 1 - percorso base

- Driver a 48 kHz, buffer 256, una sorgente ASIO IN 1 in S1 CH 1, uscita su porta libera.
- KEY MASTER OFF, Classic, PHON 50, AUTO, MASTER dB 0.
- RUN AUDIO: verificare meter e ascolto; poi STOP AUDIO.

18.2 Test 2 - gain sharing

- Due ingressi indipendenti in S1 CH 1/2, entrambi AUTO e gruppo A.
- Parlare alternativamente sui microfoni e verificare la variazione inversa di AUTO GAIN.
- Provare PHON 0, 50, 100 e ascoltare intelligibilità, code e pumping.

18.3 Test 3 - KEY virtuale

- Con un lavalier nel gruppo A, abilitare KEY MASTER e KEY A.
- Verificare che l'attenuazione in assenza di parlato cambi rispetto alla KEY disabilitata.
- Confermare che la KEY non sia udibile direttamente all'uscita.

18.4 Test 4 - stereo

- Collegare ASIO IN 1+2 (STEREO) a S1 CH 1 e una destinazione stereo fisica libera.
- Usare toni o segnali distinti su L/R e verificare che non vengano scambiati.
- Controllare stabilità dell'immagine stereo variando il parlato e osservando l'AutoGain unico.

18.5 Test 5 - catena a quattro sessioni

- Configurare S1 CH 1 ASIO, S2 CH 1 da S1 GROUP A, S3 CH 1 da S2 GROUP A e S4 CH 1 da S3 GROUP A.
- Usare solo la destinazione fisica del canale finale e lasciare spenti gli altri output hardware.
- Verificare meter di ogni sessione, eventuali attenuazioni cumulative e correttezza dei mix.

18.6 Test 6 - Sleep e Wake

- Con solo S1 collegata, controllare 1/4 AWAKE e registrare Callback Load/Peak.
- Aggiungere progressivamente i tap verso S2, S3, S4; annotare i carichi.
- Disabilitare gli ingressi delle sessioni successive e verificare il ritorno SLEEP.
- Testare un ingresso fisico valido ma silenzioso: deve rimanere ACTIVE senza perdere l'inizio di una parola.

18.7 Test 7 - stress e sicurezza

- Ridurre il buffer progressivamente a 128 e 64 campioni solo dopo il test a 256.
- Caricare fino a 32 partecipanti e verificare comportamento con sorgenti stereo, KEY, mute/solo, preset e salvataggi.
- Eseguire test prolungati, loopback fisico di latenza, monitoraggio di glitch e recovery dopo perdita del driver.

Accettazione minima per uso professionale: nessun dropout nella durata di prova stabilita, nessun cambio routing imprevisto, stabilità con il massimo carico previsto, verifica del livello di ogni uscita e percorso di bypass o fallback hardware. Questi risultati devono essere ottenuti e documentati con misure reali.

19. Prestazioni, latenza e capacità

Risorsa	Stato v0.3.1
Sessioni	4
Partecipanti per sessione	8
Partecipanti complessivi	32, condivisi nello stesso SharedPool
Gruppi di gain sharing	A/B/C per sessione
Bus di gruppo disponibili	12 tap di somma audio
Porte indicizzabili dal PATCH	Fino a 128 IN e 128 OUT; limite reale: device/driver
Dimensione massima chunk interna	1024 campioni
Lookahead del DSP	Non presente / zero campioni di latenza dichiarata al processore
Routing S1→S4	Elaborato nello stesso callback; nessun blocco di trasferimento aggiunto

La latenza round-trip va misurata inviando un impulso noto su un’uscita, riportandolo a un ingresso con loop fisico e confrontando temporalmente i due segnali registrati. Per un sistema Dante considerare anche clock, buffering DVS e latenza di rete.

Avere 32 processori istanziati non equivale a elaborare 32 flussi costantemente: Auto Session Sleep salta le sorgenti senza input utilizzabile, ma l’app mantiene strutture, timer e verifiche di routing.

20. Diagnostica dei problemi comuni

Sintomo	Verifica / intervento
Nessun audio, pulsante RUN AUDIO	Verificare dispositivo attivo, assegnazioni PATCH, ingressi disponibili e uscite libere; premere RUN AUDIO.
Audio si ferma dopo apertura PATCH	Comportamento previsto: PATCH disarma. Salvare e riarmare manualmente.
DVS/ASIO non si apre	Controllare driver, rete Dante, clock leader, sample rate, porte Ethernet e conflitti con altre app ASIO.
S2-S4 indicate SLEEP	Verificare che il tap provenga da una sessione precedente realmente attiva e abbia sorgenti valide.
Impossibile assegnare OUT 1+2	Verificare che OUT 1 e OUT 2 non siano già impegnate da singoli canali o gruppi.
Input stereo suona mono	Verificare selezione ASIO STEREO e destinazione STEREO; evitare tap MONO e conversioni tramite media.
Livello inferiore dopo più gruppi	Controllare conversioni mono↔stereo, più stadi di AutoGain, KEY e MASTER dB.
Clipping dei mix	Ridurre i livelli di sorgente/sessione; non è presente un limiter di bus.
Meters attivi, uscita silenziosa	Controllare destinazione hardware, MUTE/MASTER MUTE, canale/driver fisico e audio armato.
Callback Load vicino al 100%	Aumentare buffer, ridurre sorgenti attive, verificare driver/sistema; ascoltare dropout.
Parametri cambiati dopo riapertura	Verificare file XML della variante 4 Sleep e eventuali errori di compatibilità/percorso.

21. Limiti noti e scelte progettuali

- **Preview, non certificato:** non sono documentate misure complete di latenza, robustezza per molte ore e validazione su tutte le interfacce ASIO.
- **SharedPool distribuito:** gli score dei partecipanti non vengono campionati sincronicamente in un unico stadio centralizzato.
- **Smart sperimentale:** detector euristico, non un riconoscitore di voce né una rimozione di rumore.
- **Bus non limitati:** possibile somma oltre 0 dBFS; occorre una strategia di headroom/limiting esterna.
- **Bus mono con passaggi stereo interni:** possono verificarsi attenuazioni cumulative con conversioni mono/stereo.
- **Stato hardware:** la numerazione delle porte va verificata sui dispositivi con subset non contigui abilitati.
- **Patch modificabile a motore fermo:** non è previsto crossfade di riconfigurazione delle rotte in diretta.
- **Session Sleep:** i processori e i relativi timer esistono anche quando una sessione dorme.
- **Callback real-time:** richiede ancora audit di allocazioni e benchmark su MSVC/Windows/driver reali.

- **Non riproduzione ufficiale Dugan:** il motore trae ispirazione dal gain sharing, ma le scelte di Classic, Smart e PHON non sono specifiche ufficiali Dan Dugan.

Raccomandazione per la regia: mantenere una versione stabile di riferimento, identificare un canale di bypass hardware indipendente dall'applicazione e non sostituire un sistema automixer di produzione senza piano di validazione e rollback.

22. Glossario operativo

Termine	Definizione nel contesto SpeechShare
AutoGain	Coefficiente dinamico determinato dalla competizione fra score.
Partecipante	Una sorgente mono o stereo che prende parte al DSP come unità logica.
Weight	Moltiplicatore detector espresso in dB; non è il fader di uscita.
Strength	Esponente che accentua o riduce la selettività della competizione.
Attack / Release	Costanti temporali del movimento dei guadagni.
PHON	Macro di sessione che controlla Strength, Attack e Release.
Virtual KEY	Riferimento sintetico per il denominatore del detector; non è in uscita.
Tap	Segnale elaborato disponibile per il PATCH interno successivo.
Group bus	Somma post-processo dei partecipanti appartenenti a un gruppo.
DAG	Grafo diretto aciclico; in SpeechShare impone routing fra sessioni solo in avanti.
Auto Session Sleep	Esclusione del DSP di una sessione priva di input utilizzabili.
Callback Load	Rapporto fra tempo di calcolo del callback e durata del buffer.

23. Riferimenti ai file sorgente e verifiche

Per rendere il documento verificabile, le indicazioni tecniche si riferiscono all'archivio sorgenti v0.3.1 distribuito per il progetto. I file principali e le prove autonome sono elencati di seguito.

Area	File principali
UI e PATCH	App/Main.cpp
Engine / audio	App/StandaloneAudio.cpp · App/StandaloneAudio.h
Routing e stati	App/ModularPatch.h · App/SessionActivity.h · App/ChannelFormat.h

Area	File principali
DSP e pool	Insert/InsertDSP.cpp · Insert/SharedPool.cpp
PHON e plugin	Insert/PhonMacro.h · Insert/InsertPluginProcessor.cpp
Build e test	CMakeLists.txt · build_live_windows_2026.ps1 · Tests/*.cpp

Verifica locale di questo manuale: le sette suite di test C++ compilabili senza JUCE sono state compilate ed eseguite in ambiente Linux durante la redazione. Hanno restituito esito positivo per formato mono/stereo, routing modulare, quattro sessioni, sleep, PHON, PATCH legacy e DSP Insert.

Non verificato durante la redazione: build finale Windows della v0.3.1, apertura della sua GUI, driver ASIO specifici, assenza di allocazioni nel thread audio, misura del clock jitter, saturazione bus, profilo CPU sotto stress e comportamento in sessioni di ore.

Il manuale è un documento di **progetto** e non una dichiarazione di conformità o garanzia di idoneità per broadcast live.

24. Scheda operativa rapida

Prima di RUN AUDIO: dispositivo pronto e sincronizzato; sample rate corretto; buffer noto; porte I/O abilitate; ingressi validi; uscite esclusive e libere; KEY MASTER come previsto; MASTER dB con headroom; monitor a basso livello; fallback disponibile.

Durante il test: osservare INPUT e AUTO GAIN, controllare il percorso previsto in tutte le sessioni attraversate, verificare `x/4 AWAKE`, Callback Load e Peak; ascoltare segnali anomali, sommatorie e cross-talk.

Prima di una nuova configurazione: STOP AUDIO → PATCH / ASIO → modifica → SAVE PATCH → verifica delle collisioni → RUN AUDIO → test a basso livello.

Per la documentazione di prova: annotare versione, seriale/interfaccia, driver, sample rate, buffer, numero di canali attivi, configurazione dei gruppi, PHON, KEY, risultati della latenza misurata e anomalie riscontrate.

Fine del manuale · SpeechShare PHON Modular 4 Sleep v0.3.1 · 23 ottobre 2026